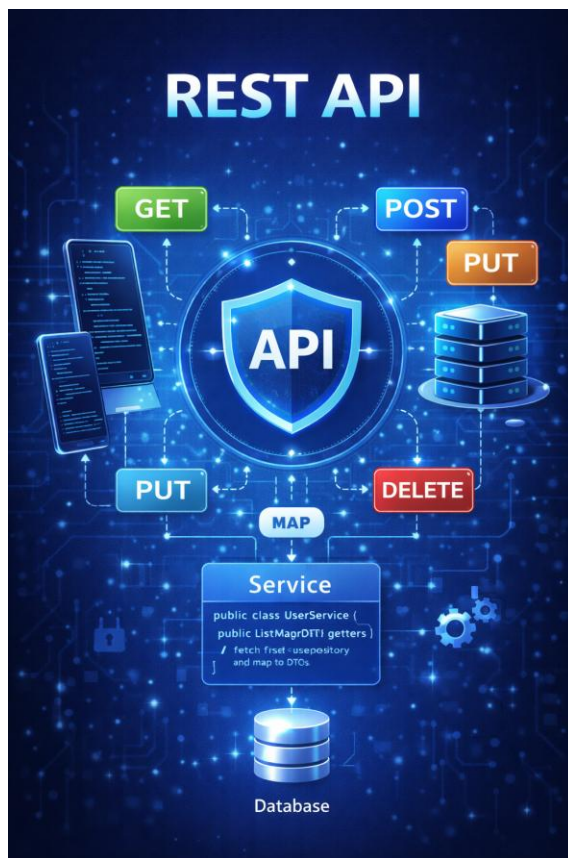




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Desarrollo de Aplicaciones Web	Unidad 2	Tema: Arquitectura de API REST
	Semana 4	Tutor: Ing. Victoria Castro

Introducción a REST:

REST (Representational State Transfer o Transferencia de Estado Representacional) no es un protocolo, sino un **estilo de arquitectura** para sistemas de software distribuidos. Fue definido por Roy Fielding en su tesis doctoral en el año 2000.

Una API REST permite que dos sistemas informáticos se comuniquen a través de Internet de forma segura y estandarizada. Se basa en la idea de que todo en la web es un **recurso** (un usuario, una imagen, un dato de clima) y que cada recurso tiene un identificador único (URI).

Sección 1:

Los 6 Principios Fundamentales (Restricciones)

Para que una API sea considerada verdaderamente **RESTful**, debe cumplir con las siguientes reglas definidas por la arquitectura original:

1. **Interfaz Uniforme:** Los recursos deben identificarse mediante URIs (ej. /api/clientes/123) y los mensajes deben ser autodescriptivos.
2. **Desacoplamiento Cliente-Servidor:** El cliente (frontend) y el servidor (backend) evolucionan de forma independiente. Al cliente no le importa cómo el servidor guarda los datos, solo cómo los solicita.
3. **Sin Estado (Stateless):** Cada petición del cliente debe contener toda la información necesaria para que el servidor la entienda. El servidor no guarda "memoria" de la petición anterior.
4. **Capacidad de Caché:** Las respuestas deben indicar si pueden ser almacenadas en caché para mejorar el rendimiento y reducir la carga del servidor.
5. **Sistema de Capas:** El cliente no sabe si está conectado directamente al servidor final o a un intermediario (como un balanceador de carga o un proxy de seguridad).
6. **Código bajo demanda (Opcional):** El servidor puede enviar código ejecutable (como scripts de Java) al cliente.

Sección 2:

Anatomía de una Petición REST

Basándonos en la documentación de **Postman**, una interacción REST exitosa se compone de:

- **Endpoint (Punto de acceso):** La URL que identifica al recurso.
- **Método HTTP:** La acción (GET, POST, PUT, DELETE).
- **Cabeceras (Headers):** Información como Content-Type: application/json o tokens de autorización.

- **Cuerpo (Body):** Los datos enviados (generalmente en formato JSON).
- **Parámetros:** * *Path parameters:* /usuarios/1 (identifica un recurso específico).
 - *Query parameters:* /usuarios?rol=admin (filtra o clasifica).

Sección 3:

Beneficios de REST según Líderes de la Industria

Escalabilidad: Al ser *stateless*, es fácil añadir más servidores sin preocuparse por sincronizar sesiones (Google Cloud).

Flexibilidad: Puede manejar múltiples tipos de formatos (JSON, XML, YAML), aunque JSON es el estándar de facto (IBM).

Portabilidad: Se puede consumir desde aplicaciones móviles, navegadores web o sistemas embebidos (AWS).

Actividad de Foro Semanal: "Conectando los Puntos"

Instrucciones: Tras haber completado las tres lecturas de esta semana (Protocolo HTTP, Patrón DTO y Arquitectura REST), responde de manera breve (máximo 10 líneas) a la siguiente pregunta en el foro:

"Imagina que eres un desarrollador y necesitas enviar los datos de un nuevo usuario desde un formulario web hacia una base de datos. Explica brevemente: ¿Qué método HTTP usarías, qué papel jugaría el DTO en este proceso y por qué decimos que esta operación sigue la arquitectura REST?"

Bibliografía

Para tu respuesta, puedes apoyarte en cualquiera de estas fuentes ya estudiadas:

- Fundamentos HTTP: [MDN Web Docs](#) y [Cloudflare](#).
- Seguridad: [AWS \(HTTP vs HTTPS\)](#).
- Arquitectura REST: [IBM](#), [Google Cloud](#) y [Red Hat](#).
- Ejemplos Prácticos: [Postman Blog](#).
- Apoyo Visual: Videos complementarios de [HTTP](#) y [Patrón DTO](#).

